

Project 4 Report: Grouping Handwritten Digits with Unsupervised Learning

The Big Takeaway: In this project, we tested whether an algorithm could organize 10,000 unlabelled handwriting samples into 10 clean groups on its own. Because it only compares where dark pixels land on a grid, it easily isolates digits with unique, simple shapes like 0 and 1. However, it regularly mixes up numbers that share the same basic strokes—like 4 and 9, or 3, 5, and 8. This shows both what simple pattern matching can do on its own and why computers need more context to truly read handwriting like a human.

1. A Different Way of Learning: No Answer Key

In our previous project, we worked with supervised learning. We gave the computer our data along with the correct answers (Buy, Sell, or Hold). The computer's only job was to learn the rules that connected the features to those known labels.

This project flips that setup completely. We handed the computer 10,000 images of handwritten numbers without any labels at all. The machine does not know that these images represent numbers, and it does not know what a 3 or an 8 looks like. Instead, it has to look at the raw pixel grids and find natural groupings on its own, based purely on how similar the images look.

2. How the Pipeline Works

1. Digitizing the Images: Every handwritten number is drawn on a 28x28 grid, giving us 784 pixels per image. We unwrap each image into a single row of 784 numbers.
2. Scaling Pixel Values: Raw ink levels go from 0 (blank white space) to 255 (solid black ink). We divide every value by 255.0 to squeeze them into a neat decimal range between 0.0 and 1.0. This keeps any single stroke from overpowering our distance calculations.
3. Running K-Means: The algorithm picks 10 center points, measures which center each image is closest to, and shifts the centers until the groups settle into place.

4. Post-Check Evaluation: We keep the true labels locked away during training. Once the model finishes sorting the images, we bring the labels back to check which numbers ended up in which piles.

Stage	What We Did	What We Learned
Part 1	Looked at raw samples and scaled pixels	Saw real handwriting differences and scaled numbers to [0.0, 1.0] for stable math.
Part 2	Tested cluster counts using the Elbow method	Found that 10 clusters provided a solid balance between compactness and simplicity.
Part 3	Extracted average images (centroids)	Clean shapes like 0 and 1 produced sharp prototypes; numbers like 4 and 9 produced blurry composites.
Part 4	Audited groups against real labels	Discovered which digits sorted cleanly and which numbers routinely mixed together.
Part 5	Squashed 784 dimensions to 2D with PCA	Confirmed visually that simple numbers drift outward while complex curved numbers crowd the center.
Part 6	Tested the model on 10 fresh, unseen numbers	Verified that typical handwriting routes cleanly to the right cluster, while tilted or messy digits drift.

3. Answering the 8 Core Project Questions

1. What did K-Means discover in the MNIST dataset?

It discovered natural visual shapes based purely on where ink landed on the page. Without knowing what digits are, it grouped numbers that share similar line angles, stroke thickness, and blank spaces.

2. What value of K did you choose, and why?

We chose $K = 10$. Our elbow test showed that jumping from 5 to 10 clusters gave us a big improvement in tightness, while expanding to 15 offered diminishing returns. Using 10 also matches our base-10 number system, letting us see if the algorithm could find all 10 digits on its own.

3. What did the cluster centroids tell you?

A centroid is the average image of everything in that group. When a centroid looked sharp and crisp (like 0 or 1), it meant nearly everyone drew the digit in the exact same spot. When it looked blurry or had ghosted lines (like a mix of 4 and 9), it showed where different handwriting styles or different digits shared overlapping strokes.

4. Did the clusters correspond closely to particular digits?

For about half of them, yes. Digits like 0, 1, 2, and 6 formed clean groups where most of the images belonged to that single digit. The other clusters ended up as mixed piles containing two or three visually similar numbers.

5. Which digits were difficult to separate?

Digits 4 and 9 gave the algorithm the most trouble because both share a tall straight line on the right and an upper box. Digits 3, 5, and 8 also mixed together frequently because all three rely on rounded bottom loops and horizontal middle strokes.

6. What did the PCA visualization show?

It showed that numbers with unique shapes (like 0 and 1) pull away toward the outer edges of the map, while numbers with complex curves crowd together in the center. It also showed that squashing 784 dimensions down to 2 forces some clusters to visually overlap on screen even when they are distinct in full space.

7. What are some limitations of using K-Means for handwritten digit images?

First, K-Means only checks whether pixel coordinates match; it does not understand connected pen strokes. Second, shifting a digit just two pixels to the right makes the algorithm treat it like a totally different shape. Third, it forces every messy or unusual scribble into one of the 10 buckets, even if it does not fit well.

8. How is this project different from Project 3 (Supervised Learning)?

In Project 3, we gave the model the answers (Buy, Sell, Hold) during training, so it learned by checking its mistakes against known targets. In Project 4, the algorithm had no answers, no hints, and no feedback. It had to discover patterns purely on its own. We only used the true labels at the end to grade how well it did.

4. Final Thoughts and Real-World Takeaways

K-Means is a fast, effective tool for finding obvious, natural splits in unlabelled data. When images have distinct visual profiles, the algorithm groups them reliably without needing any human supervision.

However, human handwriting contains natural tilt, personal style quirks, and shared strokes. Because K-Means only compares exact pixel coordinates, it cannot tell the difference between digits that overlap heavily in space. To handle those subtle differences reliably in production, you need more advanced tools—like Convolutional Neural Networks—that look at the relationship between curves, lines, and shapes rather than rigid coordinates.